

Getting started with Docker

Introduction

This tutorial will guide you to:

- Discover Docker commands
- Build an application based on a web service
- Deploy and test the application into a container (a Docker container is used)
- Scale the application into several containers (Docker compose and Swarn are used)

Installation

Docker is available in many versions and for several uses:

- Desktops
- Cloud providers
- Servers

This tutorial use the community edition.

Follow the installation procedure: <https://www.docker.com/get-started/>

Working with Docker images

Docker containers are built from Docker images. By default, Docker pulls these images from Docker Hub: <https://hub.docker.com>

Among all the images, the hub contains a basic one named hello-world:

https://hub.docker.com/_/hello-world/

To use this Docker image type this command:

```
docker run hello-world
```

The command

```
docker search
```

searches for images in the Docker hub. As an example search for the Ubuntu images with:

```
docker search ubuntu
```

Then download one of the images with:

```
docker pull ubuntu
```

Check the local images with:

`docker images`

You can then run a container in the interactive mode by combining `i` and `t` option with:

`docker run -it ubuntu`

Then interact with the Linux running inside the container with an Unix command like `"ls"`.

You can remove the image with:

`docker rmi [imageID]`

Images may have dependencies between them. In order to delete dependent images simply use:

`docker rm $(docker ps -aq)` Remove all images

`docker rmi $(docker images -q)`

while

`docker stop [Container ID]`

will stop a running container. Get the list of container using:

- `docker ps`
- `docker ps -a` Show all containers (default shows just running)

`docker start [Image ID or image name]`

will start a stopped container.

Instead of the container ID you can give a name when you run the container: `docker run -name "MyLinux" -it ubuntu`

Any Docker image can be modified. In the next section, you will install Java inside the Ubuntu container then deduce from it a new image. Start by updating Ubuntu with:

`apt-get update && apt-get upgrade`

then install Java with:

`apt-get install default-jdk`

In the end test the installation using the `java` command.

The updated container can be "saved" as a new image using `commit`:

`docker commit [ContainerID] [NewImageName]`

Fun and useful isn't it?

Publish an image to the Docker Hub

To publish an image to the Docker Hub

<https://hub.docker.com/>

an account is needed.

Retrieve the image ID on your machine: `docker images`

Tag the docker image: `docker tag imageID yourDockerHubName/imageName:version`

Example: `docker tag 1dsd512s0d myDockerID/my-service:1`

Login to docker hub:

`docker login` or

`docker login http://hub.docker.com` or

`docker login -u username -p password`

Push the image to the docker hub: `docker push yourDockerHubName/imageName:version`

Example: `docker push myDockerID/my-service:1`

Docker image creation

Java coding

Use the Spring Initializer (<https://start.spring.io/>) to create a project:

- Don't forget to choose Gradle Project
- Give it a name (Artifact)
- Add Spring Web as dependencies (library)
- Choose Java version 21

Download and unzip the project. Then open the project inside Eclipse (import gradle project) or IntelliJ / VSCode (open project).

Create a new class (along with the main program) into `src/main/java/`

```
import org.springframework.web.bind.annotation.*;
```

```
@RestController
```

```
public class HelloService {  
    @GetMapping("/")  
    public String hello() {  
        return "hello";  
    }  
}
```

```
}
```

Creation of a docker image

Images are described by a docker file. A common docker file (named Dockerfile) for a Java application is:

```
# Build stage
FROM openjdk:21-jdk-slim as builder
WORKDIR /app
COPY . .
RUN ./gradlew clean build -x test

# Runtime stage
FROM openjdk:21-jdk-slim
WORKDIR /app
COPY --from=builder /app/build/libs/*.jar app.jar

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Create the docker image using (type this command under the docker prompt):

```
docker build -t hello .
```

Check the images with:

```
docker images
```

Launch a docker container

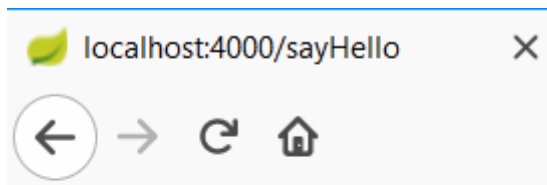
Launch a docker container with:

```
docker run -p 4000:8080 -t hello
```

You can also run the command with the option `-d` (detach mode) to run the app in the background.

This instruction tells docker to map the machine port 4000 to the container's port 8080.

Test the application in a web browser on port 4000:



Hello

If the container is running in the background (launched with `-d`) then you can interact with it using:
`docker exec -it containerID /bin/sh`

where containerID is obtained with: `docker ps`

Then list the files in the container with: `ls`

You will see the `app.jar` file: binary for the rest web service (see the Dockerfile).

Note: If you are using Docker Toolbox on Windows 7, use the Docker Machine IP instead of localhost. For example, `http://192.168.99.100:4000/`. To find the IP address, use the command `docker-machine ip`.

List the containers using:

```
docker container ls
```

Stop the container using Ctrl-C (on Windows use also "docker container stop <Container name or ID>")

Publish a Docker image in the Docker hub

Docker Hub is the world's largest library and community for container images:

<https://hub.docker.com/>

To publish your images to the Docker hub, create an account first.

Then you should tag your local image with: `docker tag imageID yourDockerHubName/imageName:version`

Example: `docker tag 1dsd512s0d efrei/my-service:1`

Then login to docker before pushing your image: `docker login`

In the end, push your image with: `docker push yourDockerHubName/imageName:version`